

Towards C++23 executors: A proposal for an initial set of algorithms

Document #: P1897R2
Date: 2020-01-10
Project: Programming Language C++ SG1
Reply-to: Lee Howes
<lwh@fb.com>

§ 1 Changelog

§ 1.1 Differences between R1 and R2

- Add `just_via` algorithm to allow type customization at the head of a work chain.
- Add `when_all` to fill missing gap in the ability to join sender chains.
- Add `indexed_for` based on feedback during the Belfast meeting to have a side-effecting algorithm.
- Propose question on replacing `bulk_execute` with `indexed_for` for the Prague meeting.

§ 1.2 Differences between R0 and R1

- Improve examples to be clearer, and fully expanded into function call form.
- Add reference to `range.adapter`.
- Remove `is_noexcept_sender`.
- Remove `just_error`.
- Clarified use of parameter packs of values and errors.
- Removed confusing use of `on` in addition to `via` in the final example.

§ 2 Introduction

In [[P0443R11](#)] we have included the fundamental principles described in [[P1660R0](#)], and the fundamental requirement to customize algorithms. In recent discussions we have converged to an understanding of the `submit` operation on a sender acting as a fundamental interoperation primitive, and algorithm customization giving us full

flexibility to optimize, to offload and to avoid synchronization in chains of mutually compatible algorithm customizations.

As a starting point, in [P0443R11] we only include a `bulk_execute` algorithm, that satisfies the core requirement we planned with [P0443R11] to provide scalar and bulk execution. To make the C++23 solution completely practical, we should extend the set of algorithms, however. This paper suggests an expanded initial set that enables early useful work chains. This set is intended to act as a discussion focus for us to discuss one by one, and to analyze the finer constraints of the wording to make sure we do not over-constrain the design.

In the long run we expect to have a much wider set of algorithms, potentially covering the full set in the current C++20 parallel algorithms. The precise customization of these algorithms is open to discussion: they may be individually customized and individually defaulted, or they may be optionally individually customized but defaulted in a tree such that customizing one is known to accelerate dependencies. It is open to discussion how we achieve this and that is an independent topic, beyond the scope of this paper.

§ 2.1 Summary

Starting with [P0443R11] as a baseline we have the following customization points:

- `execute(executor, invocable) -> void`
- `submit(sender, receiver) -> void`
- `schedule(scheduler) -> sender`
- `set_done`
- `set_error`
- `set_value`

and the following Concepts:

- `executor`
- `scheduler`
- `callback_signal`
- `callback`
- `sender`

We propose immediately discussing the addition of the following algorithms:

- `just(v)`
 - returns a sender of the value `v`
- `just_via(sch, v)`
 - a variant of the above that embeds the `via` algorithm

- `via(s, sch)`
 - returns a sender that propagates the value or error from `s` on `sch`'s execution context
- `sync_wait(s)`
 - blocks and returns the value type of the sender, throwing on error
- `when_all(s...)`
 - returns a sender that completes when all Senders `s...` complete, propagating all values
- `indexed_for(s, policy, rng, f)`
 - returns a sender that applies `f` for each element of `rng` passing that element and the values from the incoming sender, completes when all `fs` complete propagating `s`'s values onwards
- `transform(s, f)`
 - returns a sender that applies `f` to the value passed by `s`, or propagates errors or cancellation
- `bulk_transform(s, f)`
 - returns a sender that applies `f` to each element in a range sent by `s`, or propagates errors or cancellation
- `handle_error(s, f)`
 - returns a sender that applies `f` to an error passed by `s`, ignoring the values or cancellation

§ 2.2 Examples

§ 2.2.0.1 Simple example

A very simple example of applying a function to a propagated value and waiting for it.

```
auto just_sender = // sender_to<int>
    just(3);

auto transform_sender = // sender_to<float>
    transform(
        std::move(just_sender),
        [](int a){return a+0.5f;});

int result = // value: 3.5
    sync_wait(std::move(transform_sender));
```

In this very simple example we:

- start a chain with the value three
- apply a function to the incoming value, adding 0.5 and returning a sender of a float.

— block for the resulting value and assign the float value 3.5 to result.

Using operator| as in ranges to remove the need to pass arguments around, we can represent this as:

```
float f = sync_wait(
    just(3) | transform([](int a){return a+0.5f;}));
```

§ 2.2.0.2 Using indexed_for

We propose that indexed_for be the cleaned up version of bulk_execute, this shows how it fits into a work chain, with a parameter pack of inputs

```
auto just_sender = // sender_to<int>
    just(std::vector<int>{3, 4, 5}, 10);

auto indexed_for_sender = // sender_to<float>
    indexed_for(
        std::move(just_sender),
        std::execution::par,
        ranges::iota_view{3},
        [](size_t idx, std::vector<int>& vec, const int& i){
            vec[idx] = vec[idx] + i;
        });

auto transform_sender = transform(
    std::move(indexed_for_sender), [](vector<int> vec, int /*i*/)
    {return vec;});

vector<int> result = // value: {13, 14, 15}
    sync_wait(std::move(transform_sender));
```

In this less simple example we:

- start a chain with a vector of three ints and an int
- apply a function for each element in an index space, that receives the vector and int by reference and modifies the vector
- specifies the most relaxed execution policy on which we guarantee the invocable and range access function to be safe to call
- applies a transform to filter out the int result, demonstrating how indexed_for does not change the structure of the data
- block for the resulting value and assign vector {13, 14, 15} to result

Using operator| as in ranges to remove the need to pass arguments around, we can represent this as:

```
vector<int> result_vec = sync_wait(
    just(std::vector<int>{3, 4, 5}, 10) |
    indexed_for(
        std::execution::par,
```

```

        ranges::iota_view{3},
        [])(size_t idx, vector<int>&vec, const int& i){vec[idx] =
vec[idx] + i;}) |
        transform([](vector<int> vec, int /*i*/){return vec;});

```

§ 2.2.0.3 Using when_all

when_all joins a list of incoming senders, propagating their values.

```

auto just_sender =          // sender_to<int>
    just(std::vector<int>{3, 4, 5}, 10);
auto just_float_sender =    // sender_to<int>
    just(20.0f);

auto when_all_sender = when_all(
    std::move(just_sender), std::move(just_float_sender));

auto transform_sender(
    std::move(when_all_sender),
    [])(std::vector<int> vec, int /*i*/, float /*f*/) {
    return vec;
})

vector<int> result =          // value: {3, 4, 5}
    sync_wait(std::move(transform_sender));

```

This demonstrates simple joining of senders:

- start a chain with a pack of a vector and an int
- start a second chain with a float
- join the two to produce a pack of a vector, an int and a float
- applies a transform to filter out the vector result
- block for the resulting value and assign vector {3, 4, 5} to result

Using operator| as in ranges to remove the need to pass arguments around, we can represent this as:

```

vector<int> result_vec = sync_wait(
    when_all(just(std::vector<int>{3, 4, 5}, 10), just(20.0f)) |
    transform([](vector<int> vec, int /*i*/, float /*f*/){return
vec;}));

```

§ 2.2.0.4 With exception

A simple example showing how an exception that leaks out of a transform may propagate and be thrown from sync_wait.

```

int result = 0;
try {
    auto just_sender = just(3);
    auto via_sender = via(std::move(just_sender), scheduler1);

```

```

        auto transform_sender = transform(
            std::move(via_sender),
            [](int a){throw 2;});
        auto skipped_transform_sender = transform(
            std::move(transform_sender),
            [](){return 3;});

        result = sync_wait(std::move(skipped_transform_sender));
    } catch(int a) {
        result = a;
    }
}

```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1
- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value
- skip the final transform because there is an error propagating
- block for the resulting value, seeing an exception thrown instead of a value returned
- handle the exception

As before, using operator| as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```

int result = 0;
try {
    result = sync_wait(
        just(3) |
        via(scheduler1) |
        transform([](int a){throw 2;}) |
        transform([](){return 3;}));
} catch(int a) {
    result = a;
}
}

```

§ 2.2.0.5 Handle an exception

Very similar to the above, we can handle an error mid-stream

```

auto just_sender = just(3);
auto via_sender = via(std::move(just_sender), scheduler1);
auto transform_sender = transform(
    std::move(via_sender),
    [](int a){throw 2;});
auto skipped_transform_sender = transform(
    std::move(transform_sender),
    [](){return 3;});
auto error_handling_sender = handle_error(

```

```

        std::move(skipped_transform_sender),
        [] (exception_ptr e) { return just(5); });

    auto result = sync_wait(std::move(error_handling_sender));

```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1
- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value
- skip the final transform because there is an error propagating
- handle the error channel, applying an operation to an exception_ptr pointing to the value 2
- in handling the error we return a sender that propagates the value 5, thus recovering from the error
- block for the resulting value, assigning 5 to result

As before, using operator| as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```

auto s = ;
int result = sync_wait(
    just(3) |
    via(scheduler1) |
    transform([](float a){throw 2;}) |
    transform([](){return 3;}) |
    handle_error([](auto e){
        return just(5);}));

```

§ 3 Impact on the standard library

§ 3.1 Sender adapter objects

Taking inspiration from [range adaptors](#) define sender adapters.

Wording to be based on [range.adaptors] with the basic requirement that:

- operator| be overloaded for the purpose of creating pipelines over senders
- That the following are equivalent expressions:
 - algorithm(sender, args...)
 - algorithm(args...)(sender)
 - sender | algorithm(args...)
- that algorithms(args...) is a *sender adaptor closure object*

— TBD where sender adapters are declared

Details below are in loosely approximated wording and should be made consistent with [P0443R11] and the standard itself when finalized. We choose this set of algorithms as a basic set to allow a range of realistic, though still limited, compositions to be written against executors.

§ 3.2 **execution::just**

§ 3.2.1 **Overview**

`just` creates a sender that propagates a value inline to a submitted receiver.

Signature:

```
S<T...> just(T...);
```

where `S<T...>` is an implementation-defined `typed_sender` that that sends a set of values of type `T...` in its value channel.

[Example:

```
int r = sync_wait(just(3));  
// r==3
```

- end example]

§ 3.2.2 **Wording**

The expression `execution::just(t...)` returns a sender, `s` wrapping the values `t...`.

- If `t...` are nothrow movable then `execution::is_noexcept_sender(s)` shall be `constexpr` and return `true`.
- When `execution::submit(s, r)` is called for some `r`, and `r`-value `s` will call `execution::set_value(r, std::move(t)...)`, inline with the caller.
- When `execution::submit(s, r)` is called for some `r`, and `l`-value `s` will call `execution::set_value(r, t...)`, inline with the caller.
- If moving of `t` throws, then will catch the exception and call `execution::set_error(r, e)` with the caught `exception_ptr`.

§ 3.3 **execution::just_via**

§ 3.3.1 **Overview**

`just_via` creates a sender that propagates a value to a submitted receiver on the execution context of a passed scheduler. Semantically equivalent to `just(t) | via(s)` if `just_via` is not customized on `s`.

Signature:

```
S<T...> just_via(Scheduler, T...);
```

where `S<T...>` is an implementation-defined `typed_sender` that that sends a set of values of type `T...` in its value channel in the context of the passed `Scheduler`.

[Example:

```
MyScheduler s;  
int r = sync_wait(just_via(s, 3));  
// r==3
```

- end example]

§ 3.3.2 Wording

The name `execution::just_via` denotes a customization point object. The expression `execution::just_via(sch, t...)` for some subexpression `S` is expression-equivalent to:

- `sch.just(t...)` if that expression is valid.
- Otherwise, `just_via(sch, t...)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class Sch, class T...>  
void just_via(Sch, T...) = delete;
```

and that does not include a declaration of `execution::just_via`.

- Otherwise returns the result of the expression: `via(just(t...), sch)`

§ 3.4 execution::sync_wait

§ 3.4.1 Overview

Blocks the calling thread to wait for the passed sender to complete. Returns the value (or void if the sender carries no value), throws if an exception is propagated and throws a TBD exception type on cancellation.¹ On propagation of the `set_done()` signal, returns an empty optional.

```
T... sync_wait(S<T...>)
```

where `S<T...>` is a sender that sends zero or one values of type `T...` in its value channel. The existence of, and if existing the type `T` must be known statically and cannot be part of an overload set.

[Example:

```
int r = sync_wait(just(3));  
// r==3
```

- end example]

§ 3.4.2 Wording

The name `execution::sync_wait` denotes a customization point object. The expression `execution::sync_wait(S)` for some subexpression `S` is expression-equivalent to:

- `S.sync_wait()` if that expression is valid.
- Otherwise, `sync_wait(S)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S>
    void sync_wait(S) = delete;
```

and that does not include a declaration of `execution::sync_wait`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`. Waits on the synchronization primitive to block on completion of `S`.
 - If `set_value` is called on `r`, returns the passed value (or simply returns for void sender).
 - If `set_error` is called on `r`, throws the error value as an exception.
 - If `set_done` is called on `r`, throws some TBD cancellation exception type.

§ 3.5 `execution::via`

§ 3.5.1 Overview

`via` is a sender adapter that takes a sender and a scheduler and returns a sender that propagates the same value as the original, but does so on the scheduler's execution context.

Signature:

```
S<T...> via(S<T...>, Scheduler);
```

where `S<T>` is an implementation-defined type that is a sender that sends a value of type `T` in its value channel.

[Example:

```
static_thread_pool t{1};
int r = sync_wait(just(3) | via(t.scheduler()));
// r==3
```

§ 3.5.2 Wording

The name `execution::via` denotes a customization point object. The expression `execution::via(S, Sch)` for some subexpressions `S`, `Sch` is expression-equivalent to:

- `S.via(Sch)` if that expression is valid.
- Otherwise, `via(S, Sch)` if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class Sch>
void via(S, Sch) = delete;
```

- Otherwise constructs a receiver `r` such that when `set_value`, `set_error` or `set_done` is called on `r` the value(s) or error(s) are packaged, and a receiver `r2` constructed such that when `execution::set_value(r2)` is called, the stored value or error is transmitted and `r2` is submitted to `Sch`. If `set_error` or `set_done` is called on `r2` the error or cancellation is propagated and the packaged values ignored.
- The returned sender's value types match those of `sender1`.
- The returned sender's execution context is that of `scheduler1`.

§ 3.6 `execution::when_all`

§ 3.6.1 Overview

`when_all` combines a set of *non-void* senders `s`, returning a sender that, on success, completes with the combined values of all incoming senders.

Signature:

```
S<T0..., T1..., Tn...> when_all(S<Tn...>);
```

where `S<T>` is an implementation-defined type that is a sender that sends a value of type `T` in its value channel.

[Example:

```
float r =
    sync_wait(
        transform(
            when_all(just(3) | just(1.2f)),
            [](int a, float b){return a + b;});
    // r==4.2
```

§ 3.6.2 Wording

The name `execution::when_all` denotes a customization point object. The expression `execution::when_all(S)` for some subexpression `S` is expression-equivalent to:

- Otherwise, `when_all(S)` if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S>
    void when_all(S) = delete;
```

- Otherwise constructs a receiver, r_i for each passed sender S_i in S and passes that receiver to `execution::submit( $S_i$ ,  $r_i$ )`. When some `output_receiver` has been passed to `submit` on the returned sender.
 - if `set_value( $t\dots$ )` is called on all r_i , will concatenate the list of values and call `set_value( $output\_receiver$ ,  $t_0\dots$ ,  $t_1\dots$ ,  $t_n\dots$ )` on the receiver passed to `submit` on the returned sender.
 - if `set_done()` is called on any r_i , will call `set_done( $output\_receiver$ )`, discarding other results.
 - if `set_error( $e$ )` is called on any r_i will call `set_error( $output\_receiver$ ,  $e$ )` for some e , discarding other results.

§ 3.7 execution::indexed_for

§ 3.7.1 Overview

`indexed_for` is a sender adapter that takes a sender, execution policy, a range and an invocable and returns a sender that propagates the input values but runs the invocable once for each element of the range, passing the input by non-const reference.

Signature:

```
S<T...> indexed_for(
    S<T...>,
    execution_policy,
    range<Idx>,
    invocable<void(Idk, T&...)>);
```

where `S<T...>` represents implementation-defined sender types that send a value of type `T...` in their value channels. Note that in the general case there may be many types `T...` for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
int r = sync_wait(
    just(3) |
    indexed_for(
        std::execution::par,
        ranges::iota_view{6},
        [](int idx, int& v){v = v + idx;});
// r==9
```

§ 3.7.2 Wording

The name `execution::indexed_for` denotes a customization point object. The expression `execution::indexed_for( $S$ ,  $P$ ,  $R$ ,  $F$ )` for some subexpressions S , P , R and F is

expression-equivalent to:

- If `P` does not satisfy `std::is_execution_policy_v<P>`, then the expression is invalid.
- If `R` does not satisfy either range then the expression is invalid.
- If `P` is `std::execution::sequenced_policy` then range must satisfy `input_range` otherwise range must satisfy `random_access_range`.
- If `F` does not satisfy `MoveConstructible` then the expression is invalid.
- `S.indexed_for(P, R, F)`, if that expression is valid.
- Otherwise, `indexed_for(S, R, P, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class R, class P, class F>
void indexed_for(S, R, P, F) = delete;
```

and that does not include a declaration of `execution::indexed_for`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.
 - If `set_value` is called on `r` with some parameter pack `t...` then calls `F(idx, t...)` for each element `idx` in `R`. Once all complete calls `execution::set_value(output_receiver, v)`.
 - If any call to `set_value` throws an exception, then call `set_error(r, e)` with some exception from the set.
 - If `set_error(r, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.
 - If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

Notes: * If `P` is not `execution::seq` and `R` satisfies `random_access_range` then `indexed_for` may run the instances of `F` concurrently. * `P` represents a guarantee on the most relaxed execution policy `F` and the element access function of range `R` are safe to run under, and hence the most parallel fashion in which the underlying scheduler may map instances of `F` to execution agents.

§ 3.8 execution::transform

§ 3.8.1 Overview

`transform` is a sender adapter that takes a sender and an invocable and returns a sender that propagates the value resulting from calling the invocable on the value passed by the preceding sender.

Signature:

```
S<T2> transform(S<T...>, invocable<T2(T...)>);
```

where $S<T...>$ and $S<T2>$ are implementation-defined types that represent senders that send a value of type list $T...$ or $T2$ respectively in their value channels. Note that in the general case there may be many types $T...$ for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
int r = sync_wait(just(3) | transform([](int v){return v+1;}));  
// r==4
```

§ 3.8.2 Wording

The name `execution::transform` denotes a customization point object. The expression `execution::transform(S, F)` for some subexpressions S and F is expression-equivalent to:

- $S.transform(F)$ if that expression is valid.
- Otherwise, `transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>  
void transform(S, F) = delete;
```

and that does not include a declaration of `execution::transform`.

- Otherwise constructs a receiver, r over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`. When some `output_receiver` has been passed to `submit` on the returned sender.
 - If `set_value(r, Ts... ts)` is called, calls `std::invoke(F, ts...)` and passes the result v to `execution::set_value(output_receiver, v)`.
 - If F throws, catches the exception and passes it to `execution::set_error(output_receiver, e)`.
 - If `set_error(c, e)` is called, passes e to `execution::set_error(output_receiver, e)`.
 - If `set_done(c)` is called, calls `execution::set_done(output_receiver)`.

§ 3.9 execution::bulk_transform

§ 3.9.1 Overview

`bulk_transform` is a sender adapter that takes a sender of a range of values and an invocable and returns a sender that executes the invocable for each element of the input range, and propagates the range of returned values.

Signature:

```
S<range<T2>> bulk_transform(S<range<T>>, invocable<T2(T)>);
```

where `S<range<T>>` and `S<T2>` are implementation-defined types that represent senders that send a value of type list `T` or `T2` respectively in their value channels. Note that in the general case there may be many types `T` for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
std::vector<int> r = sync_wait(just(std::vector<int>{3, 4, 5}) |
bulk_transform([](int v){return v+1;}));
// r=={4, 5, 6}
```

Note: it is TBD how precisely we should represent the intermediate data types here. Intermediate vectors would require allocator support. Purely lazy ranges may be inadequate.

§ 3.9.2 Wording

The name `execution::bulk_transform` denotes a customization point object. The expression `execution::bulk_transform(S, F)` for some subexpressions `S` and `F` is expression-equivalent to:

- `S.bulk_transform(F)`, if that expression is valid.
- Otherwise, `bulk_transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
void bulk_transform(S, F) = delete;
```

and that does not include a declaration of `execution::bulk_transform`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.
 - If `S::value_type` does not model the concept `Range<T>` for some `T` the expression ill-formed.
 - If `set_value` is called on `r` with some parameter input applies the equivalent of `out = std::ranges::transform_view(input, F)` and passes the result output to `execution::set_value(output_receiver, v)`.
 - If `set_error(r, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.
 - If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

§ 3.10 execution::handle_error

§ 3.10.1 Overview

`handle_error` is a sender adapter that takes a sender and an invocable and returns a sender that propagates the value, error or done signal from the sender returned by the invocable.

Signature:

```
S<T2..., E2...> handle_error(S<T..., E...>,
    invocable<sender<T2..., E2...>(E...)>);
```

where $S<T\ldots, E\ldots>$ and $S<T2\ldots, E2\ldots>$ are implementation-defined types that represent senders that send a value of type list $T\ldots$ or $T2\ldots$ respectively in their value channels and error type lists $E\ldots$ and $E2\ldots$ in their error channels. The invocable takes the error types $E\ldots$ and returns a sender over some potentially new set of types. By returning a sender the algorithm has control of error recovery as well as use cases such as logging and propagation. Note that in the general case there may be many types $E\ldots$ for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
float r = sync_wait(
    just(3) |
    transform([](int v){throw 2.0f;}) |
    handle_error([](float e){return just(e+1);}));
// r==3.0f
```

§ 3.10.2 Wording

The name `execution::handle_error` denotes a customization point object. The expression `execution::handle_error(S, F)` for some subexpressions S and F is expression-equivalent to:

- $S.handle_error(F)$, if that expression is valid.
- Otherwise, `handle_error(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
    void handle_error(S, F) = delete;
```

and that does not include a declaration of `execution::handle_error`.

- Otherwise constructs a receiver, r over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.
- If `set_value(r, v...)` is called, passes $v\ldots$ to `execution::set_value(output_receiver, v...)`.
- If `set_error(r, e...)` is called, passes $e\ldots$ to f , resulting in a sender $s2$ and passes `output_receiver` to `submit(s2, output_receiver)`.
- If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

§ 4 Customization and example

Each of these algorithms, apart from `just`, is customizable on one or more sender implementations. This allows full optimization. For example, in the following simple work chain:

```
auto s = just(3) | // s1
        via(scheduler1) | // s2
        transform([](int a){return a+1;}) | // s3
        transform([](int a){return a*2;}) | // s4
        via(scheduler2) | // s5
        handle_error([](auto e){return just(3);}); // s6
int r = sync_wait(s);
```

The result of `s1` might be a `just_sender<int>` implemented by the standard library vendor.

`via(just_sender<int>, scheduler1)` has no customization defined, and this expression returns an `scheduler1_via_sender<int>` that is a custom type from the author of `scheduler1`, it will call `submit` on the result of `s1`.

`s3` calls `transform(scheduler1_via_sender<int>, [](int a){return a+1;})` for which the author of `scheduler1` may have written a customization. The `scheduler1_via_sender` has stashed the value somewhere and build some work queue in the background. We do not see `submit` called at this point, it uses a behind-the-scenes implementation to schedule the work on the work queue. An `scheduler1_transform_sender<int>` is returned.

`s4` implements a very similar customization, and again does not call `submit`. There need be no synchronization in this chain.

At `s5`, however, the implementor of `scheduler2` does not know about the implementation of `scheduler1`. At this point it will call `submit` on the incoming `scheduler1_transform_sender`, forcing `scheduler1`'s sender to implement the necessary synchronization to map back from the behind-the-scenes optimal queue to something interoperable with another vendor's implementation.

`handle_error` at `s6` will be generic in terms of `submit` and not do anything special, this uses the default implementation in terms of `submit`. `sync_wait` similarly constructs a `condition_variable` and a temporary `int`, submits a receiver to `s` and waits on the `condition_variable`, blocking the calling thread.

`r` is of course the value 8 at this point assuming that neither scheduler triggered an error. If there were to be a scheduling error, then that error would propagate to `handle_error` and `r` would subsequently have the value 3.

§ 5 Potential future changes

§ 5.1 bi-directional via

via will become a bi-directional algorithm. It will propagate a scheduler upstream as discussed in [P1898R0]. It will switch context to the passed scheduler, and allow customization of the returned receiver as discussed above.

§ 5.2 when_all's context

Based on experience in Facebook's codebase, I believe that `when_all` should return a sender that requires an executor-provider and uses forward progress delegation as discussed in [P1898R0]. The returned sender should complete on the delegated context. This removes the ambiguity about which context it completes on.

§ 5.3 when_all for void types and mixed success

We should add a `when_all` variant that returns tuples and variants in its result, or some similar mechanism for to allow parameter packs, including empty packs in the form of void-senders, and mixed success/error to propagate.

§ 6 Proposed question for the Prague 2020 meeting

§ 6.1 Replace `bulk_execute` in P0443 with `indexed_for` as described above.

`indexed_for` as described above should replace `bulk_execute` during the merge of [P0443R11] into C++23. Suggest fine-tuning this wording and forwarding to LEWG.

The changes this leads to:

- Renames the algorithm to fit in with a set of user-level algorithms rather than making it distinct and internal-only. We found it hard to define a difference between `bulk_execute` and `indexed_for` and so suggest we not try, instead we rename it.
- Propagating the data from the incoming sender into the invocable by reference and out the other end. This allows the algorithm to be a side-effecting view on data, but because that data is in-band in the data stream it is safe from a lifetime point of view. More so that it would be if the data had to be captured by reference.
- Replaces the max value with a range for the index space. This allows for more flexibility.
- Adds the execution policy back in, defining the forward progress guarantee both the invocable and range accessor make. This is preferred because the policy is a statement the programmer makes about the capabilities of the invocable. An `indexed_for` that requires `seq`, and an executor that cannot execute `seq` can fail at this

point. An invocable that requires seq run on an executor that cannot run seq algorithms would be invisible at the point of chaining the algorithm.

- Does not add any sort of factory as [P1993R0]. These are not necessary if we carry the data in the stream. Data can be allocated to a device using, for example, a `device_vector`. This maintains full flexibility - we can add custom data management algorithms independently and keep `indexed_for` focused on its primary use cause: the asynchronous for loop itself.
- Relaxes the CopyConstructible restriction that [P0443R11], but also the standard algorithms in C++20 place. Wide discussion suggests that this restriction may not be necessary, and it could certainly be harmful. In an asynchronous world we cannot rely on scoped `reference_wrapper` semantics, and the cost of injecting `shared_ptr` would be high. If an implementation needs to copy, then that implementation should implement a wrapper that is custom for the algorithmic structure it is using. For example, a forking tree of threads may allocate once on the first thread by move and reference back to it, knowing the lifetime is safe.

§ 7 References

- [P0443R11] 2019. A Unified Executors Proposal for C++.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0443r11.html>
- [P1660R0] 2019. A Compromise Executor Design Sketch.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1660r0.pdf>
- [P1898R0] 2019. Forward progress delegation for executors.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1898r0.html>
- [P1993R0] 2019. Restore factories to `bulk_execute`.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1993r0.pdf>

-
1. Other options include an optional return type.↩